

Análisis UI/UX y Arquitectura de Blog y Eventos

Ministerio REDES · Informe técnico y de experiencia de usuario · 20 de septiembre de 2026

Alcance. Auditoría del flujo público y administrativo, librerías utilizadas, componentes propios, persistencia, contratos API, estados de carga/error, accesibilidad, riesgos y oportunidades de mejora.

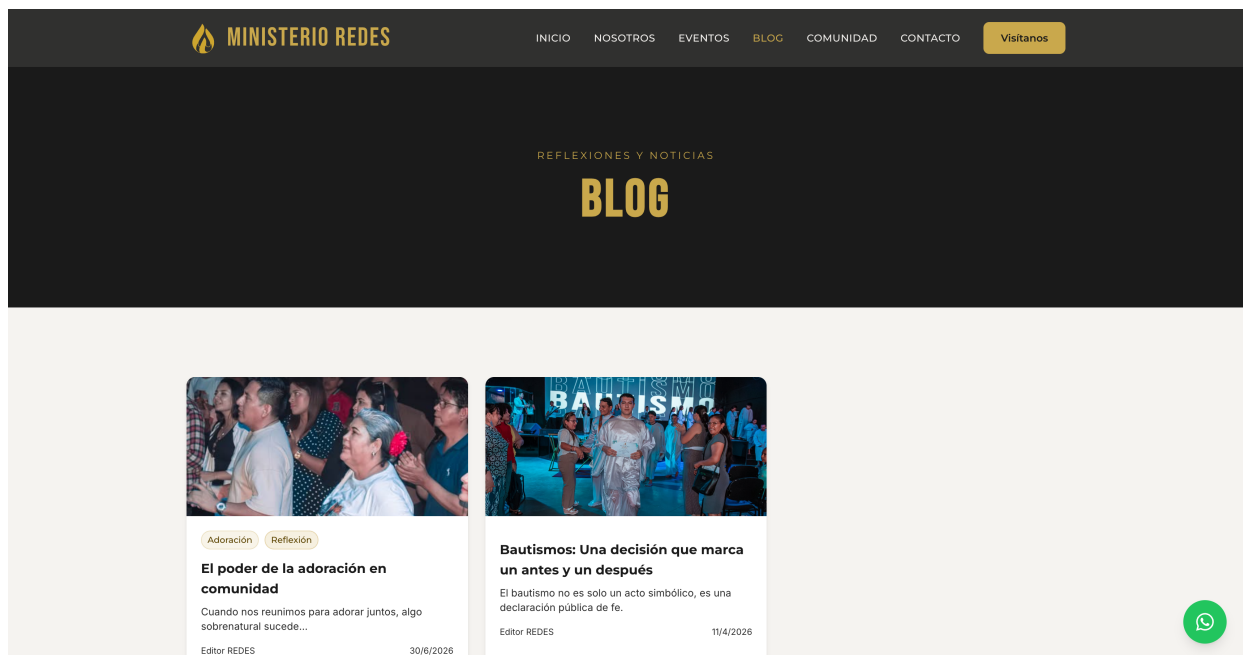


Figura 1. Listado público del Blog.

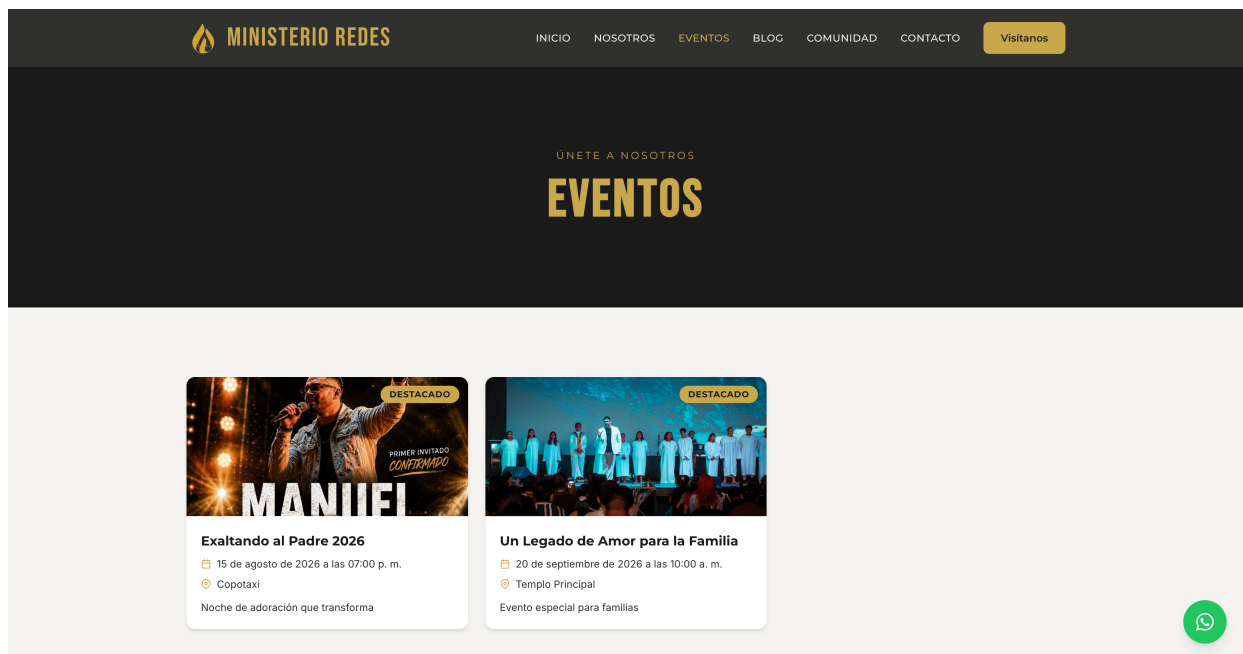


Figura 2. Listado público de Eventos.

1. Resumen ejecutivo

Blog y Eventos están implementados como dos dominios CRUD independientes que comparten la misma infraestructura: React/Vite en el cliente, Axios para HTTP, Express/TypeScript en el servidor, Prisma como ORM y PostgreSQL como persistencia. El portal público consume únicamente publicaciones publicadas y eventos próximos/en curso; el panel administrativo expone el ciclo completo de creación, edición, publicación y eliminación según rol.

El flujo de Eventos es funcional y relativamente directo. El flujo de Blog es más rico: incorpora BlockNote, conversión HTML, portada multimedia, etiquetas, estados editoriales, SEO y botones de compartir. Durante la auditoría se reprodujo un fallo real al abrir un artículo: el editor quedaba en blanco por una incompatibilidad entre React 18 y Mantine 9.5.2. Se corrigió alineando Mantine 8 con React 18 y se verificó el editor en navegador.

Evaluación resumida

Área	Estado actual	Riesgo / oportunidad
Eventos público	Funcional, tarjetas y detalle	Falta paginación visible, filtros y mapa/registro más contextual
Eventos admin	Formulario claro con validación básica	Persistencia de fechas y campos opcionales necesita validación de servidor más estricta
Blog público	Tarjetas, tags, portada, detalle y compartir	Contenido HTML se inserta con dangerouslySetInnerHTML; requiere sanitización explícita
Blog admin	Editor BlockNote, portada, tags, publicación y SEO	Complejidad alta; dependencia de editor sensible a versiones
Accesibilidad	Labels, botones y estados principales presentes	Hay advertencias de labels/ids en Blog editor y controles del sistema
Arquitectura	Separación clara por rutas y servicios	Axios duplicado en páginas admin; falta capa API/admin compartida

Hallazgo corregido durante la auditoría

Blog no cargaba al pulsar Editar. La consola mostraba `render2 is not a function` dentro de `MantineProvider`. `@mantine/core@9.5.2` declara React 19 como peer dependency, mientras el proyecto usa React 18. Se ajustaron `@mantine/core` y `@mantine/hooks` a 8.3.x; el editor volvió a montar correctamente.

2. Arquitectura general

El frontend se monta como una SPA única. Las rutas públicas y administrativas viven en el mismo árbol React, pero el prefijo `/admin` se renderiza fuera del Layout público.

```
frontend/src/App.tsx
/admin/* -> AdminApp lazy + AuthGuard
/ -> Layout público + páginas Home, About, Blog, Events, Community, Contact
```

El servidor Express registra las rutas públicas y administrativas sobre el mismo router de dominio. La protección se aplica a las rutas administrativas mediante JWT y roles.

```
backend/src/index.ts
/api/v1/events -> event.routes.ts
/api/v1/posts -> blog.routes.ts
/api/v1/admin/events -> event.routes.ts protegido por middleware
/api/v1/admin/posts -> blog.routes.ts protegido por middleware
```

Librerías relevantes

Librería	Uso en Blog/Eventos	Observación UX/técnica
React + React Router	Render, rutas y formularios	Navegación SPA sin recarga; rutas de edición por ID

Librería	Uso en Blog/Eventos	Observación UX/técnica
Vite	Dev server y build	HMR acelerado; requiere reinicio cuando cambia optimización de dependencias
Axios	CRUD y carga de datos	Cada página admin crea su propia instancia y interceptor JWT
Framer Motion	Animación de tarjetas y detalle	Entrada progresiva; no bloquea el contenido
BlockNote	Editor enriquecido del Blog	Editor por bloques, serializado a HTML; sensible a compatibilidad Mantine/React
Mantine	UI interna de BlockNote	Debe mantenerse alineada con la versión de React del proyecto
react-icons	Iconografía de acciones y metadatos	Consistente con el lenguaje visual del panel
MediaPicker / MediaUpload	Portadas de posts y flyers	Dos caminos: elegir biblioteca o subir a Cloudinary
react-share	Compartir artículos	Facebook, X y WhatsApp en detalle de Blog
Prisma	Persistencia y consultas	Servicios de dominio encapsulan la mayoría de reglas CRUD

3. Módulo de Eventos

Eventos tiene dos experiencias: el catálogo público y el backoffice de gestión. El dominio de datos está representado por el modelo Prisma Event.

Flujo público

- Events.tsx solicita hasta 20 eventos mediante `eventsApi.getAll(1, 20)`.
- El backend filtra estados UPCOMING y ONGOING y ordena por `startDate` ascendente.
- La tarjeta muestra flyer, etiqueta Destacado, título, fecha, ubicación y `shortDescription`.
- El enlace navega a `/eventos/:slug` y `EventDetail.tsx` obtiene el detalle por slug.
- `EventDetail` muestra hero, fecha, ubicación, flyer, descripción, capacidad, dirección y `registrationUrl`.
- `EventImage` centraliza el fallback/selección visual del flyer.

Archivos principales de Eventos

Archivo	Responsabilidad
<code>frontend/src/pages/Events.tsx</code>	Listado público, loading skeleton, empty state, tarjetas y metadatos
<code>frontend/src/pages/EventDetail.tsx</code>	Detalle por slug, error de no encontrado, CTA de registro
<code>frontend/src/admin/pages/events/EventList.tsx</code>	Tabla admin, estado, fecha, flyer, editar y eliminar
<code>frontend/src/admin/pages/events/EventForm.tsx</code>	Crear/editar, fechas, ubicación, capacidad, estado, flyer y registro
<code>frontend/src/components/events/EventImage.tsx</code>	Presentación/fallback de imágenes de eventos
<code>backend/src/routes/event.routes.ts</code>	Rutas públicas y admin con orden de rutas para evitar conflictos
<code>backend/src/services/event.service.ts</code>	Consultas, slugify, estados, fechas, creación, actualización y eliminación
<code>backend/prisma/schema.prisma</code>	Modelo Event, EventStatus y relaciones con User

Formulario administrativo de Eventos

- Título y fecha de inicio son obligatorios; la validación ocurre en cliente antes de enviar.
- La imagen puede seleccionarse con `MediaPicker` o subirse con `MediaUpload`.
- El estado usa DRAFT, UPCOMING, ONGOING, COMPLETED y CANCELLED.
- `isFeatured` controla la insignia Destacado y el endpoint featured del catálogo público.
- El backend convierte fechas a `Date` y `latitud/longitud` a `Decimal/número` cuando existen.
- Al cambiar el título, el backend recalcula el slug sin comprobar explícitamente colisiones en `update`.

REDES

Panel de Administración

Dashboard

Eventos

Blog

Media

Configuración

Acerca del Sistema

A

Cerrar sesión

Dashboard

Ver sitio

← Editar Evento

Título *

Un Legado de Amor para la Familia

Descripción corta

Evento especial para familias

Descripción completa

Fortalece los lazos familiares bajo la guía de Dios. Actividades para toda la familia.

Flyer / Imagen



Estado

Próximo

☒ Evento destacado

Detalles

Fecha de inicio *

20/09/2026, 03:00 PM

Fecha de fin

20/09/2026, 09:00 PM

Ubicación

Templo Principal

Dirección

20 de Junio y Cotopaxi, Lago Agrio, Ecuador

Figura 3. Formulario administrativo de edición de Eventos.

Ministerio REDES - Auditoría UI/UX de Blog y Eventos

Página 5

4. Módulo de Blog

Blog es el módulo con mayor densidad funcional. Combina contenido editorial enriquecido, taxonomía, publicación, SEO, portada multimedia y distribución social.

Flujo público

- Blog.tsx consulta `postsApi.getAll()` y solo recibe publicaciones PUBLISHED desde el backend.
- Las tarjetas muestran portada, tags, título, excerpt, autor y fecha publicada.
- BlogPost.tsx obtiene un artículo por slug y construye metadata SEO con `Seo.tsx`.
- El contenido se renderiza como HTML mediante `dangerouslySetInnerHTML`.
- `react-share` genera botones para Facebook, X/Twitter y WhatsApp.
- Si no existe portada, se muestra un fallback visual con REDES.

Archivos principales de Blog

Archivo	Responsabilidad
<code>frontend/src/pages/Blog.tsx</code>	Listado público y tarjetas de artículos
<code>frontend/src/pages/BlogPost.tsx</code>	Detalle, portada, HTML, metadata y compartir
<code>frontend/src/admin/pages/blog/PostList.tsx</code>	Listado admin, publicación/despublicación, editar y eliminar
<code>frontend/src/admin/pages/blog/PostEditor.tsx</code>	Editor BlockNote, portada, tags, estado y SEO
<code>frontend/src/admin/components/editor/</code>	Bloques, slash menu, YouTube, columnas y media con texto
<code>frontend/src/admin/components/upload/MediaUpload.tsx</code>	Subida con metadata/label y Cloudinary vía API
<code>backend/src/routes/blog.routes.ts</code>	CRUD público/admin, detalle y tags
<code>backend/src/services/blog.service.ts</code>	Slug, publicación, tags, author, consultas y CRUD
<code>backend/prisma/schema.prisma</code>	BlogPost, Tag y PostTag

Editor administrativo

- `useCreateBlockNote` construye el editor con `BlockNoteSchema` y el bloque `custom mediaWithText`.
- `HTMLToBlocks` convierte el contenido persistido a bloques al editar.
- `createInternalHTMLSerializer` convierte los bloques a HTML antes de guardar.
- Se eliminan clases/UI administrativa de bloques `media-with-text` antes de persistir.
- La portada puede elegirse de `MediaPicker` o cargarse mediante `MediaUpload`.
- Tags se cargan desde `/posts/tags` y se envían como `tagIds`.
- El estado editorial se controla con botones Borrador, Publicado y Archivado.
- SEO title y description tienen límites 60/160 y contador visual.

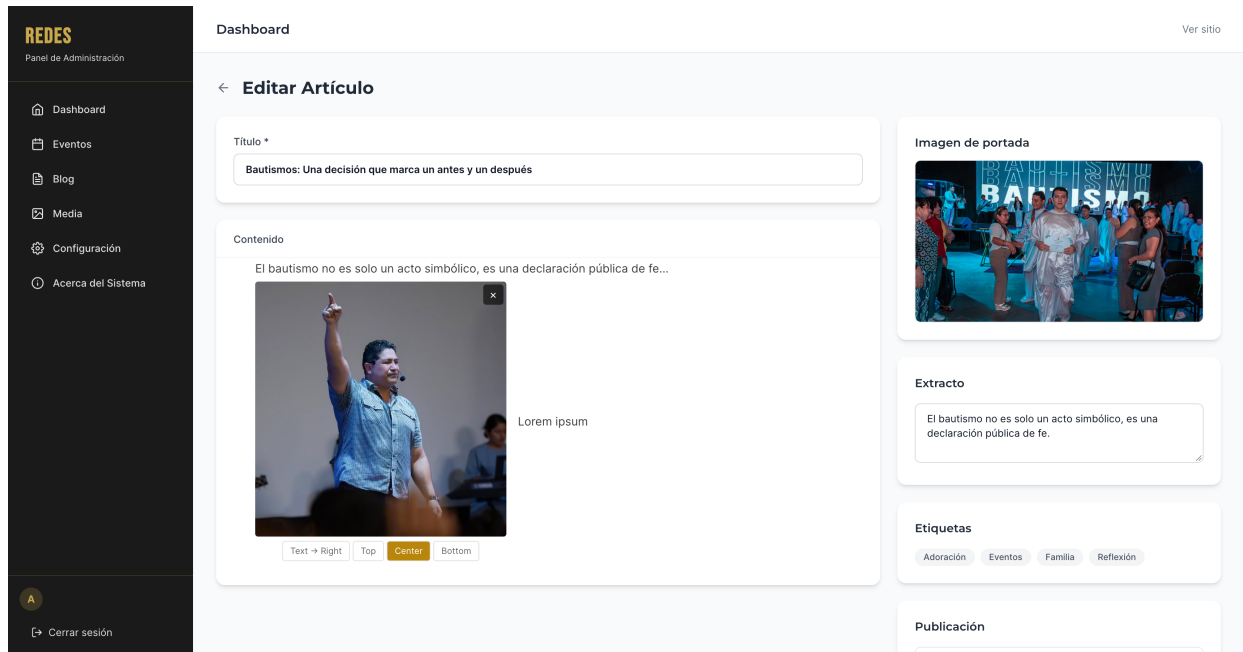


Figura 4. Editor administrativo de Blog después de corregir la incompatibilidad de Mantine.

5. Persistencia y API

Los dos dominios usan un patrón de rutas delgadas y servicios de dominio. Las rutas controlan autorización y formato HTTP; los servicios ejecutan Prisma y reglas específicas.

Operación	Eventos	Blog
Listado público	GET /api/v1/events	GET /api/v1/posts
Detalle público	GET /api/v1/events/:slug	GET /api/v1/posts/:slug
Listado admin	GET /api/v1/admin/events/all	GET /api/v1/admin/posts/all
Detalle admin	GET /api/v1/admin/events/detail/:id	GET /api/v1/admin/posts/detail/:id
Crear	POST /api/v1/admin/events	POST /api/v1/admin/posts
Actualizar	PUT /api/v1/admin/events/:id	PUT /api/v1/admin/posts/:id
Eliminar	DELETE /api/v1/admin/events/:id	DELETE /api/v1/admin/posts/:id
Autorización	ADMIN/EVENT_MANAGER	ADMIN/EDITOR

Modelos de datos

```
Event
title, slug, description, shortDescription, startDate, endDate
location, address, latitude, longitude, flyerUrl, gallery
status, isFeatured, capacity, registrationUrl, createdById

BlogPost
title, slug, content, excerpt, coverImageUrl
authorId, status, publishedAt, readTime, seoTitle, seoDescription, ogImageUrl

Tag / PostTag
relación N:M entre artículos y etiquetas
```

Persistencia de imágenes

MediaUpload recibe el archivo en memoria mediante multer, lo envía a Cloudinary con transformaciones de calidad/formato y persiste URLs original, thumbnail y medium en Media. El editor de Blog y el formulario de Eventos pueden usar también MediaPicker para seleccionar una imagen existente.

La biblioteca exige actualmente una etiqueta al subir. La tabla Media contiene label, altText, dimensiones, tamaño y usuario cargador. Esta consistencia fue reparada mediante una migración que agregó la columna label a bases existentes.

6. Evaluación UI/UX

Fortalezas

- El panel separa claramente listado y formulario mediante rutas /new y /:id.
- Las acciones principales tienen jerarquía consistente: Nuevo, Editar, Publicar, Eliminar.
- Los estados se expresan con colores y etiquetas legibles.
- El editor de Blog agrupa portada, extracto, tags, publicación y SEO en tarjetas.
- Eventos usa una estructura de formulario familiar: contenido principal, sidebar y detalles.
- Las tarjetas públicas priorizan imagen, título y metadatos; la información se puede escanear rápidamente.
- Existe feedback de loading, empty state, toast y confirmación de eliminación.
- Los botones de edición tienen aria-labels descriptivos en las listas.

Problemas y riesgos

Prioridad	Hallazgo	Impacto	Recomendación
Alta	Blog usa dangerouslySetInnerHTML	Riesgo XSS si HTML no confiable llega al frontend	Sanitizar con allowlist en backend y/o DOMPurify antes de renderizar
Alta	Errores del editor podían dejar la pantalla en blanco	Pérdida total del flujo de edición	Mantener versiones React/Mantine/BlockNote bloqueadas y añadir smoke test de ruta :id
Alta	Campos del Blog no tienen ids/labels completos	Problemas de accesibilidad y automatización	Asociar label htmlFor con id estable en título, BlockNote, SEO y tags
Media	Axios se instancia en cada página admin	Headers y errores duplicados	Crear un cliente admin único y servicios tipados
Media	Errores de carga redirigen silenciosamente a la lista	El usuario no sabe si falló auth, red o recurso	Mostrar alerta contextual y diferenciar 401/404/500
Media	Fechas se muestran en timezone local sin explicación	Riesgo de discrepancia Ecuador/servidor	Normalizar timezone y mostrar zona en el formulario
Media	El endpoint público de detalle no filtra estado	Un slug de borrador podría ser accesible	Restringir detalle público a PUBLISHED o introducir preview autenticado
Baja	Eventos no tienen filtros/paginación visible	Escalabilidad UX limitada	Añadir filtros por estado, fecha y destacado
Baja	Blog no muestra paginación aunque API la entrega	Contenido antiguo queda difícil de descubrir	Implementar paginador o carga incremental

Accesibilidad

- La tabla admin necesita caption o un heading asociado para contexto de lector de pantalla.
- Los iconos de acción están correctamente rotulados en listas, pero algunos botones del formulario de Evento no tienen aria-label.
- El editor BlockNote genera controles complejos: debe probarse navegación por teclado y foco tras cargar un artículo.
- La tarjeta pública depende de color para Destacado/estado; el texto también está presente, lo cual es positivo.
- Los warnings observados de labels/ids en PostEditor deben resolverse antes de auditoría WCAG formal.

7. Recomendaciones de arquitectura

Corto plazo

- Añadir un cliente HTTP admin compartido con interceptor JWT, normalización de errores y timeouts.
- Crear pruebas de navegación que abran /admin/dashboard/events/:id y /admin/dashboard/blog/:id con datos reales simulados.
- Sanitizar HTML de Blog al guardar y al renderizar.
- Validar duplicados de slug también en updateEvent y updatePost.
- No ocultar errores de carga redirigiendo silenciosamente; mostrar mensaje y acción de reintento.

Mediano plazo

- Extraer formularios en subcomponentes: EventDetails, EventMedia, PostContent, PostTaxonomy, PostSeo.
- Introducir schemas de validación compartidos, por ejemplo Zod, en frontend y backend.
- Usar respuestas tipadas para eventos, posts, tags y paginación.
- Mover serialización BlockNote/HTML a un módulo especializado y probado.
- Agregar preview autenticado del Blog antes de publicar.
- Añadir auditoría de cambios editoriales: quién, cuándo, qué campos y estado anterior.

Largo plazo

- Separar contenido editorial y configuración de página en contratos explícitos.
- Incorporar versionado o borradores con preview para Blog y Eventos.
- Optimizar imágenes y generar srcset responsive desde Cloudinary.
- Adoptar observabilidad: métricas de API, errores de editor, latencia de media y fallos de publicación.

8. Conclusión

La arquitectura actual es coherente para un CMS institucional pequeño: los dominios están bien delimitados, el público y el admin comparten contratos claros y la experiencia visual mantiene una identidad consistente. Eventos es simple y estable; Blog ofrece capacidades editoriales avanzadas, pero necesita más disciplina de dependencias, seguridad HTML y accesibilidad. La corrección de Mantine/React resolvió el bloqueo de edición observado y debe quedar protegida por pruebas de navegación y un lockfile estable.

Capturas incluidas: listado público de Blog, listado público de Eventos, editor admin de Blog y editor admin de Eventos. Los archivos originales se conservan en `.artifacts/`.